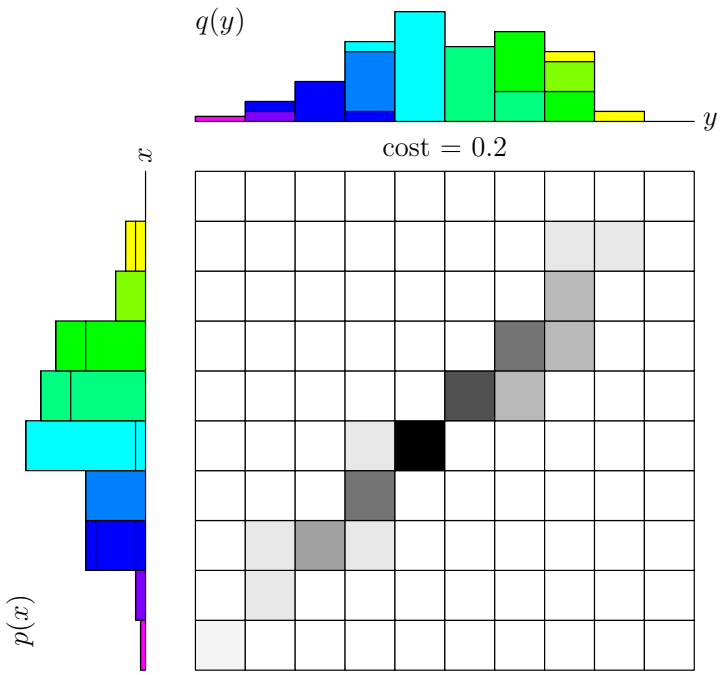
Mathematics of Machine Learning

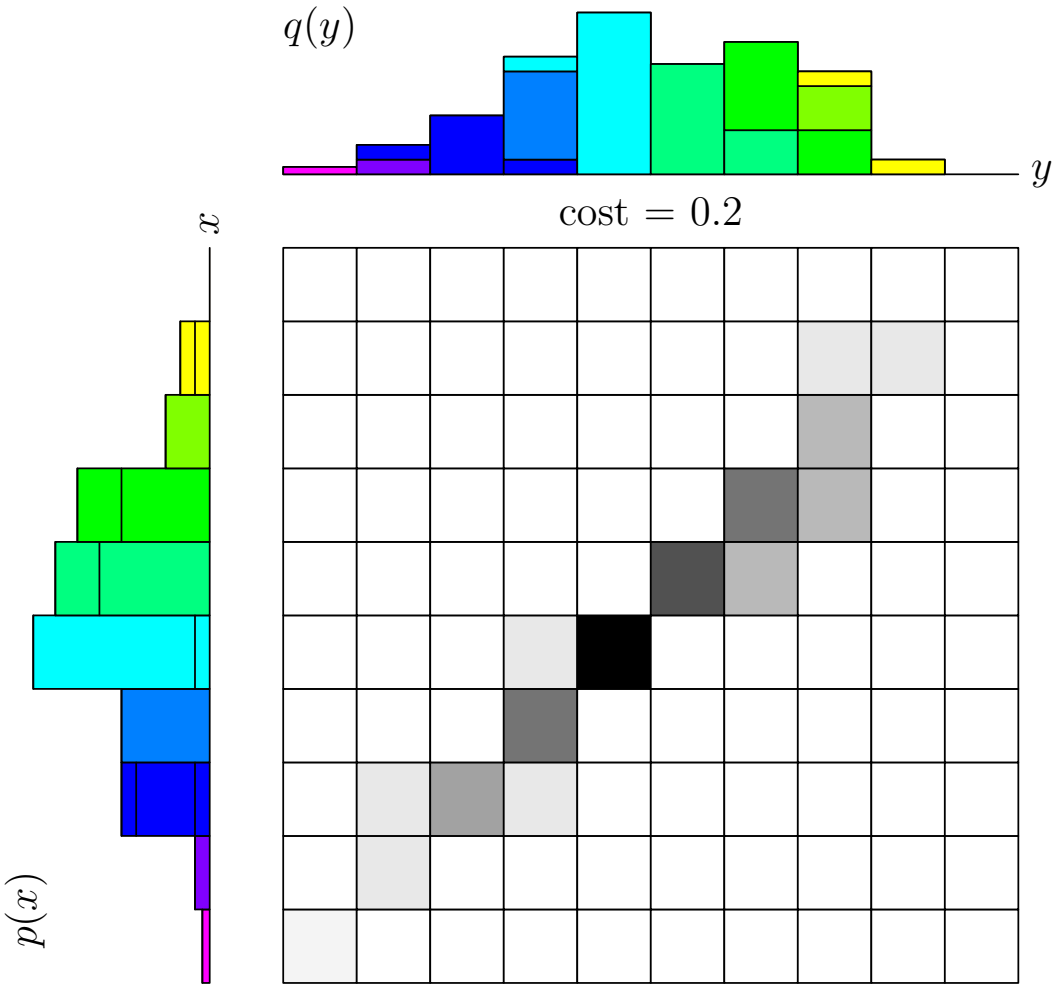
Optimal Transport and the Wasserstein Distance



Wasserstein distance, optimal transport, duality, Lipschitz functions, WGANs, Sinkhorn, FID

Outline

1. **Comparing Distributions**
2. Optimal Transport
3. Duality
4. Applications



Measuring Distances Between Distributions

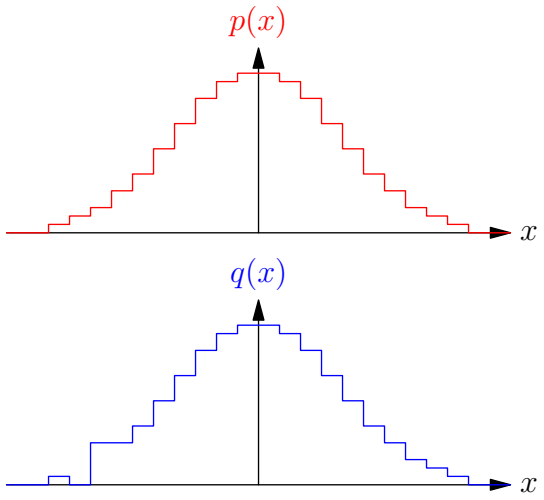
- In many machine learning tasks we want to minimise the distance between two probability distributions (e.g. make a generative model's samples look like the data)
- This requires that we can measure distances between probability distributions
- One prominent measure is the KL divergence (last lecture)

$$\mathrm{KL}(p\|q) = \int p(\boldsymbol{x}) \log\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right) \mathrm{d}\boldsymbol{x}$$

- This is very commonly used in ML (e.g. cross-entropy loss, VAEs, variational approximation)

Trouble with KL

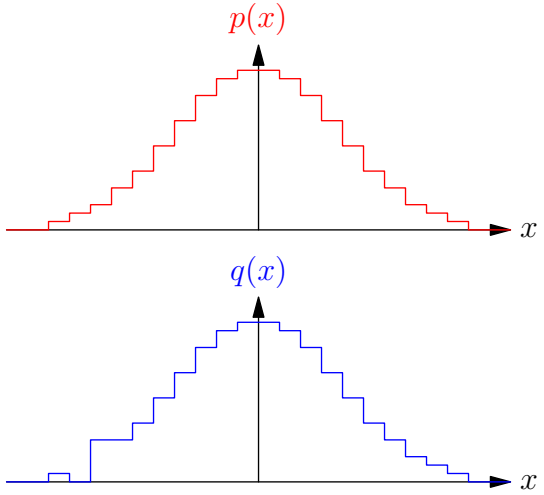
- KL-divergences are non-negative quantities that are minimised when the two probability distributions are the same
- They are not distances (they aren't symmetric and they don't satisfy the triangular inequality)
- We don't really care about this, but what we do care about is that if $q(\boldsymbol{x}) = 0$ when $p(\boldsymbol{x}) \neq 0$ then $\log\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right)$ diverges



- We can therefore have distributions that seem very similar but their KL-divergence is huge (or infinite)

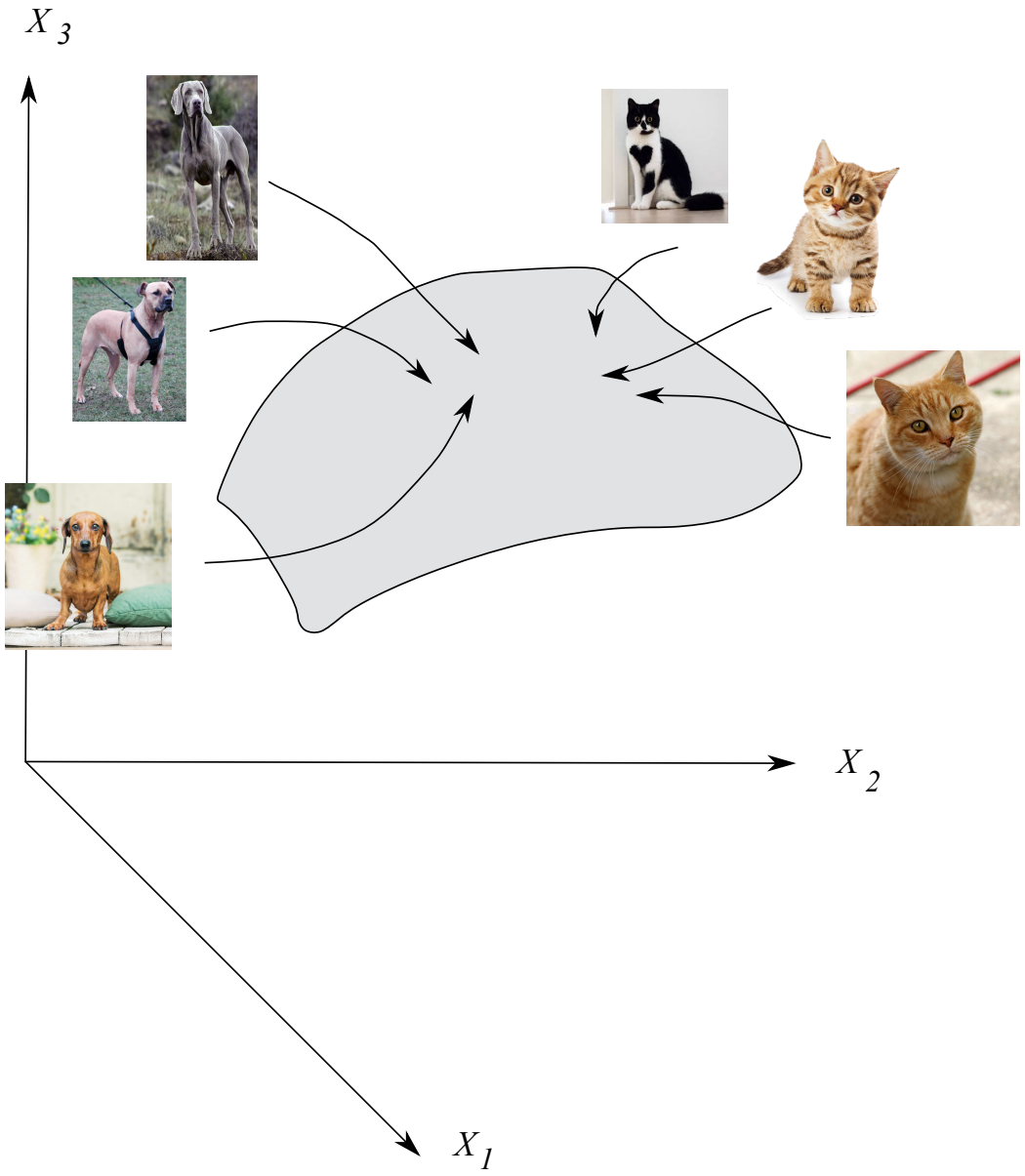
Wasserstein Distance

- A more benign measure of the differences between two probability distributions is the **Wasserstein** or **Earth Mover's** distance
- This is a true distance, but more importantly, it measures distance in a very natural way so that distributions that are close have a small Wasserstein distance



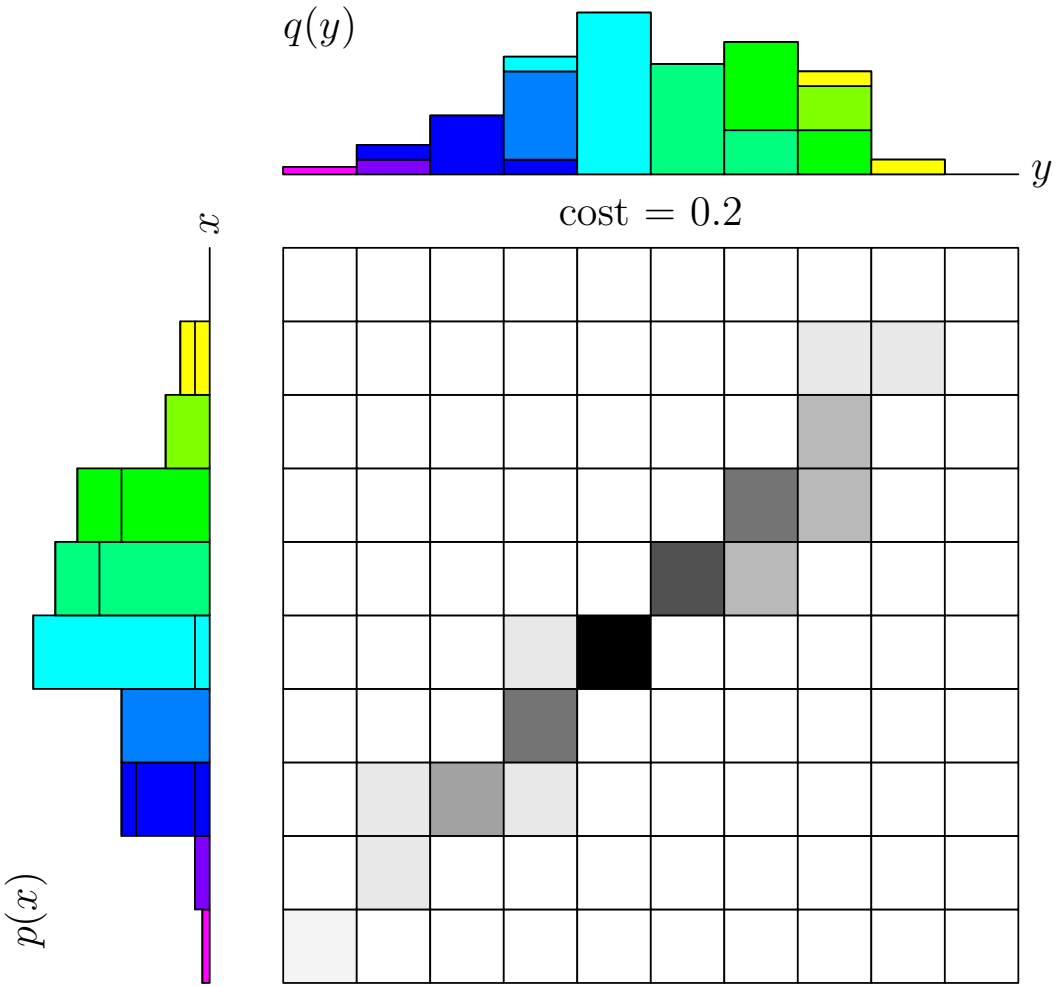
- This may seem contrived, but images (e.g. $128 \times 128 \times 3$ arrays of pixel values) lie close to a thin manifold, so two distributions of images can be close without overlapping: then the KL-divergence is useless but the Wasserstein distance is not

High Probability Manifold



Outline

1. Comparing Distributions
2. **Optimal Transport**
3. Duality
4. Applications



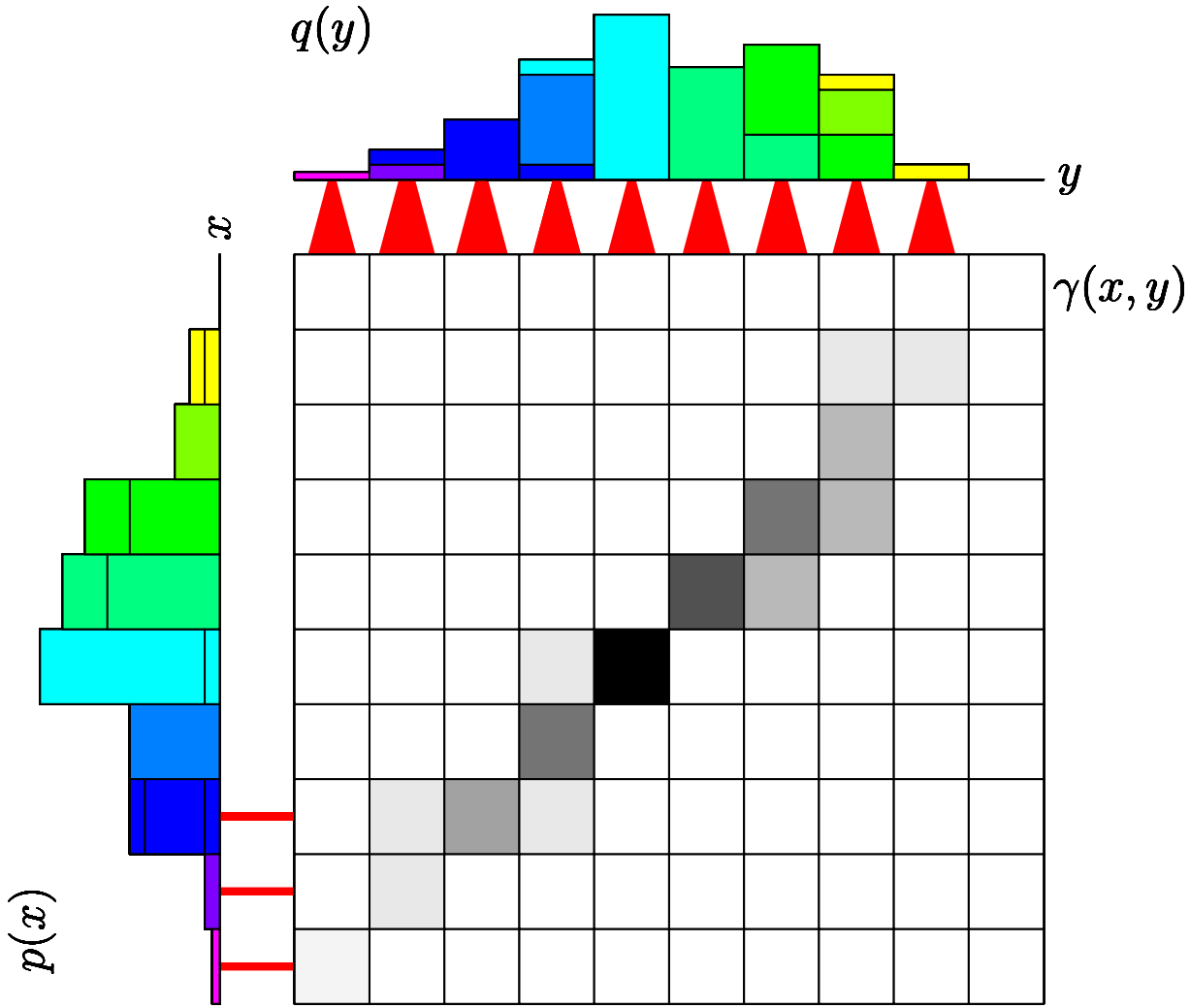
Transportation Policy

- But how do we formalise the Wasserstein distance?
- A good place to start is to define a transportation policy $\gamma(\boldsymbol{x},\boldsymbol{y})$ with

$$\int \gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{y} = p(\boldsymbol{x}) \qquad \int \gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{x} = q(\boldsymbol{y})$$

- This looks like a joint probability distribution, but we interpret $\gamma(\boldsymbol{x},\boldsymbol{y})$ as the amount of probability mass/density that we transfer from $p(\boldsymbol{x})$ to $q(\boldsymbol{y})$

Transportation Policy



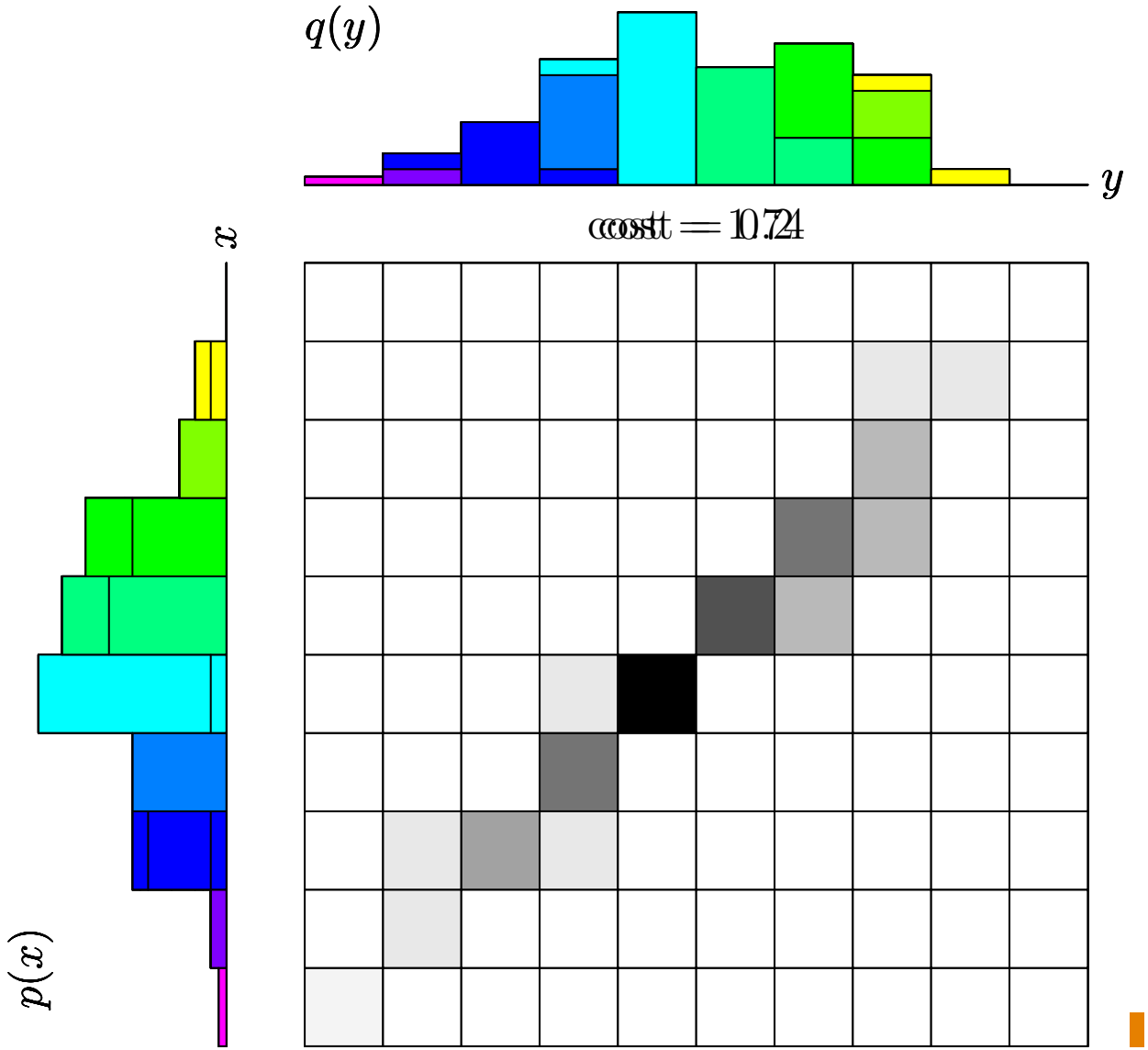
The Cost of Transport

- We want the transportation policy that minimises the total work: the probability mass moved times the distance it is moved
- Let $d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x}-\boldsymbol{y}\|$ be a distance measure then the cost of a transportation policy is

$$C(\gamma) = \int\int d(\boldsymbol{x},\boldsymbol{y})\gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} = \mathbb{E}_\gamma[d(\boldsymbol{x},\boldsymbol{y})]$$

  where we interpret $\gamma(\boldsymbol{x},\boldsymbol{y})$ as a probability distribution
- Usually we take $d(\boldsymbol{x},\boldsymbol{y})$ to be the Euclidean distance, but we can choose any distance

Transportation Cost



The Wasserstein Distance

- The Wasserstein distance $W(p,q)$ between two probability distributions is defined as

$$W(p,q) = \min_{\gamma \in \Lambda(p,q)} \mathbb{E}_\gamma[d(\boldsymbol{x},\boldsymbol{y})]$$

- Where $\Lambda(p,q)$ is the set of joint distributions $\gamma(\boldsymbol{x},\boldsymbol{y})$ such that

$$\int \gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{y} = p(\boldsymbol{x}) \qquad \int \gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{x} = q(\boldsymbol{y})$$

- With cost d^k we get $W_k(p,q) = \left(\min_\gamma \mathbb{E}_\gamma\left[d(\boldsymbol{x},\boldsymbol{y})^k\right]\right)^{1/k}$; here we use $W = W_1$

One Dimension

- In one dimension the Wasserstein distance has a closed form in terms of the cumulative distribution functions F_p and F_q

$$W(p,q) = \int_{-\infty}^{\infty} \left| F_p(x) - F_q(x) \right| \mathrm{d}x$$

  the area between the two curves

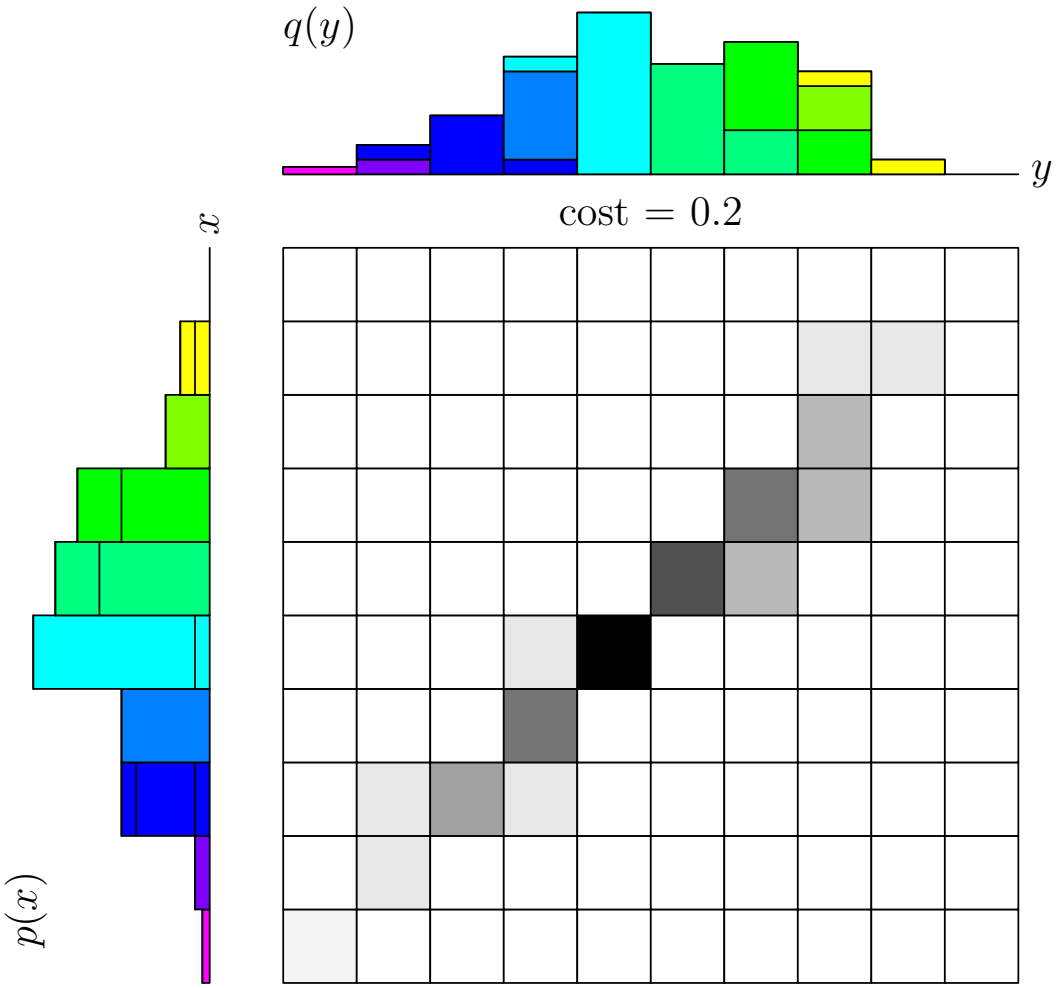
- For two samples of the same size: sort both and match them in order

$$W = \frac{1}{n} \sum_{i=1}^{n} \left| x_{(i)} - y_{(i)} \right|$$

- In higher dimensions there is no such shortcut: we have to solve the optimisation

Outline

1. Comparing Distributions
2. Optimal Transport
3. **Duality**
4. Applications



Computing the Wasserstein Distance

- To compute the Wasserstein distance we have to solve a minimisation task!
- This looks nasty, but it is a (continuous) linear programme, like those in the constrained-optimisation lecture
- Suppose p and q were discrete distributions (i.e. $\boldsymbol{x}$ and $\boldsymbol{y}$ only take discrete points)
- Then we could treat each value of $\gamma(\boldsymbol{x},\boldsymbol{y})$ as an element of a vector $\boldsymbol{\gamma}$ and each value of $d(\boldsymbol{x},\boldsymbol{y})$ as an element of a vector $\boldsymbol{D}$
- Our objective is to choose $\boldsymbol{\gamma}$ to minimise $\boldsymbol{D}^{\mathsf{T}}\boldsymbol{\gamma}$

Constraints

$$\sum_j \gamma(\boldsymbol{x}_i, \boldsymbol{y}_j) = p(\boldsymbol{x}_i) \qquad \sum_i \gamma(\boldsymbol{x}_i, \boldsymbol{y}_j) = q(\boldsymbol{y}_j)$$

$$\mathbf{A}\boldsymbol{\gamma} = \boldsymbol{P}$$

$$\begin{pmatrix}
1 & 1 & \cdots & 1 & 0 & 0 & \cdots & 0 & \cdots & 0 & 0 & \cdots & 0 \\
0 & 0 & \cdots & 0 & 1 & 1 & \cdots & 1 & \cdots & 0 & 0 & \cdots & 0 \\
\vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots & \cdots & \vdots & \vdots & \cdots & \vdots \\
0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & \cdots & 1 & 1 & \cdots & 1 \\
1 & 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & \cdots & 1 & 0 & \cdots & 0 \\
0 & 1 & \cdots & 0 & 0 & 1 & \cdots & 0 & \cdots & 0 & 1 & \cdots & 0 \\
\vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots & \cdots & \vdots & \vdots & \cdots & \vdots \\
0 & 0 & \cdots & 1 & 0 & 0 & \cdots & 1 & \cdots & 0 & 0 & \cdots & 1
\end{pmatrix}
\begin{pmatrix}
\gamma(x_1, y_1) \\ \gamma(x_2, y_1) \\ \vdots \\ \gamma(x_n, y_1) \\ \gamma(x_1, y_2) \\ \gamma(x_2, y_2) \\ \vdots \\ \gamma(x_n, y_2) \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \gamma(x_1, y_n) \\ \gamma(x_2, y_n) \\ \vdots \\ \gamma(x_n, y_n)
\end{pmatrix}
=
\begin{pmatrix}
q(y_1) \\ q(y_2) \\ \vdots \\ q(y_n) \\ p(x_1) \\ p(x_2) \\ \vdots \\ p(x_n)
\end{pmatrix}$$

The Dual Problem

- With Lagrange multipliers $\alpha(\boldsymbol{x}_i)$ and $\beta(\boldsymbol{y}_j)$ for the two sets of constraints

$$\mathcal{L} = \sum_{i,j} d(\boldsymbol{x}_i, \boldsymbol{y}_j)\gamma(\boldsymbol{x}_i, \boldsymbol{y}_j) - \sum_i \alpha(\boldsymbol{x}_i)\left(\sum_j \gamma(\boldsymbol{x}_i, \boldsymbol{y}_j) - p(\boldsymbol{x}_i)\right) - \sum_j \beta(\boldsymbol{y}_j)\left(\sum_i \gamma(\boldsymbol{x}_i, \boldsymbol{y}_j) - q(\boldsymbol{y}_j)\right)$$

  subject to $\gamma(\boldsymbol{x}_i, \boldsymbol{y}_j) \geq 0$

- Rearranging

$$\mathcal{L} = \sum_i \alpha(\boldsymbol{x}_i)p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)q(\boldsymbol{y}_j) - \sum_{i,j} \gamma(\boldsymbol{x}_i, \boldsymbol{y}_j)(\alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) - d(\boldsymbol{x}_i, \boldsymbol{y}_j))$$

- This is equivalent to maximising $\sum_i \alpha(\boldsymbol{x}_i)p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)q(\boldsymbol{y}_j)$, subject to

$$\forall i, j \quad \alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) \leq d(\boldsymbol{x}_i, \boldsymbol{y}_j)$$

- For continuous distributions sums become integrals and $\alpha(\boldsymbol{x})$, $\beta(\boldsymbol{y})$ become functions

Dual Form Constraint

- We need $\alpha(\boldsymbol{x}) + \beta(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y})$ for all $\boldsymbol{x}$, $\boldsymbol{y}$, and want β as large as possible
- Taking $\boldsymbol{x} = \boldsymbol{y}$ (where $d(\boldsymbol{y},\boldsymbol{y}) = 0$) gives $\beta(\boldsymbol{y}) \leq -\alpha(\boldsymbol{y})$; so $-\alpha$ is the largest β could possibly be
- Try that largest value, $\beta = -\alpha$: the constraint becomes

$$\alpha(\boldsymbol{x}) - \alpha(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y}) \quad \text{for all } \boldsymbol{x},\boldsymbol{y}$$

- For any α that satisfies this, $\beta = -\alpha$ is allowed and cannot be beaten
- Restricting to such α loses nothing (the **Kantorovich–Rubinstein** theorem; see the notes)

Kantorovich–Rubinstein Duality

- Putting this together

$$W(p,q) = \max_{\alpha} \int \alpha(\boldsymbol{x}) \big(p(\boldsymbol{x}) - q(\boldsymbol{x})\big) \,\mathrm{d}\boldsymbol{x}$$
$$= \max_{\alpha} \Big(\mathbb{E}_p[\alpha(\boldsymbol{X})] - \mathbb{E}_q[\alpha(\boldsymbol{X})] \Big)$$

- where the maximum is over functions with

$$\alpha(\boldsymbol{x}) - \alpha(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$$

  that is, α is **1-Lipschitz** (Lipschitz-1)

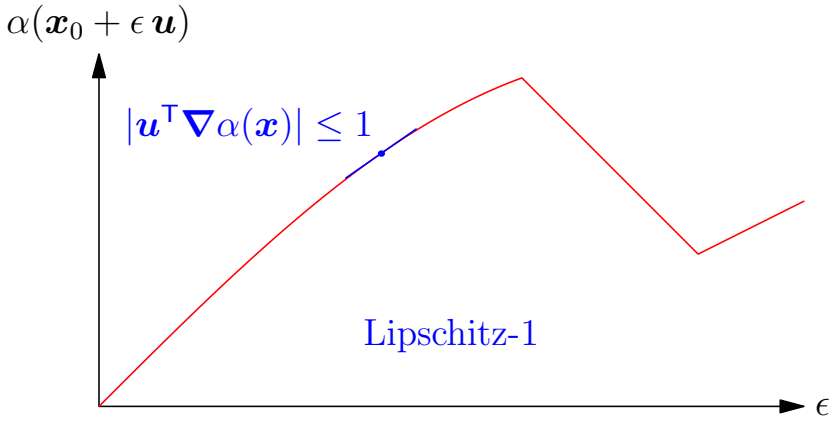
- Instead of the huge object $\gamma(\boldsymbol{x},\boldsymbol{y})$ we now need one function $\alpha(\boldsymbol{x})$, and the expectations can be estimated from samples

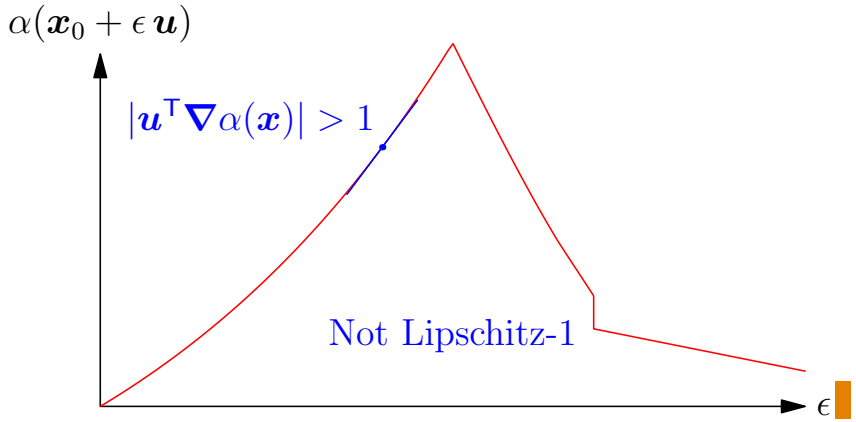
Lipschitz-1 Functions

- We note for a Lipschitz-1 function and any unit vector $\boldsymbol{u}$

$$\boldsymbol{u}^\mathsf{T}\boldsymbol{\nabla}\alpha(\boldsymbol{x}) = \lim_{\epsilon\to 0}\frac{\alpha(\boldsymbol{x}+\epsilon\boldsymbol{u})-\alpha(\boldsymbol{x})}{\epsilon} \leq \frac{\epsilon\|\boldsymbol{u}\|}{\epsilon} = \frac{\epsilon}{\epsilon} = 1$$

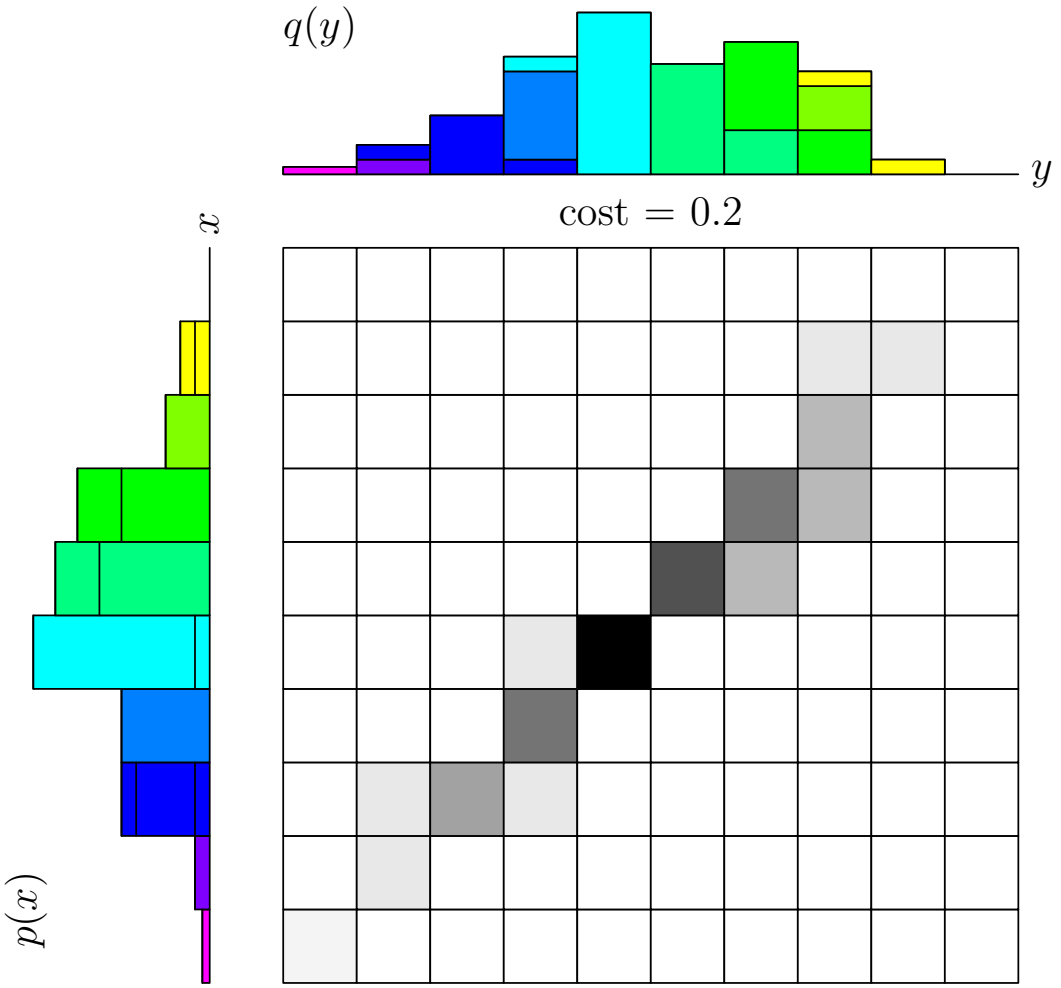
- That is, at every point the gradient in every direction is at most 1 (since the gradient defines the direction of greatest increase it is both necessary and sufficient for $\|\boldsymbol{\nabla}\alpha(\boldsymbol{x})\| \leq 1$ everywhere)



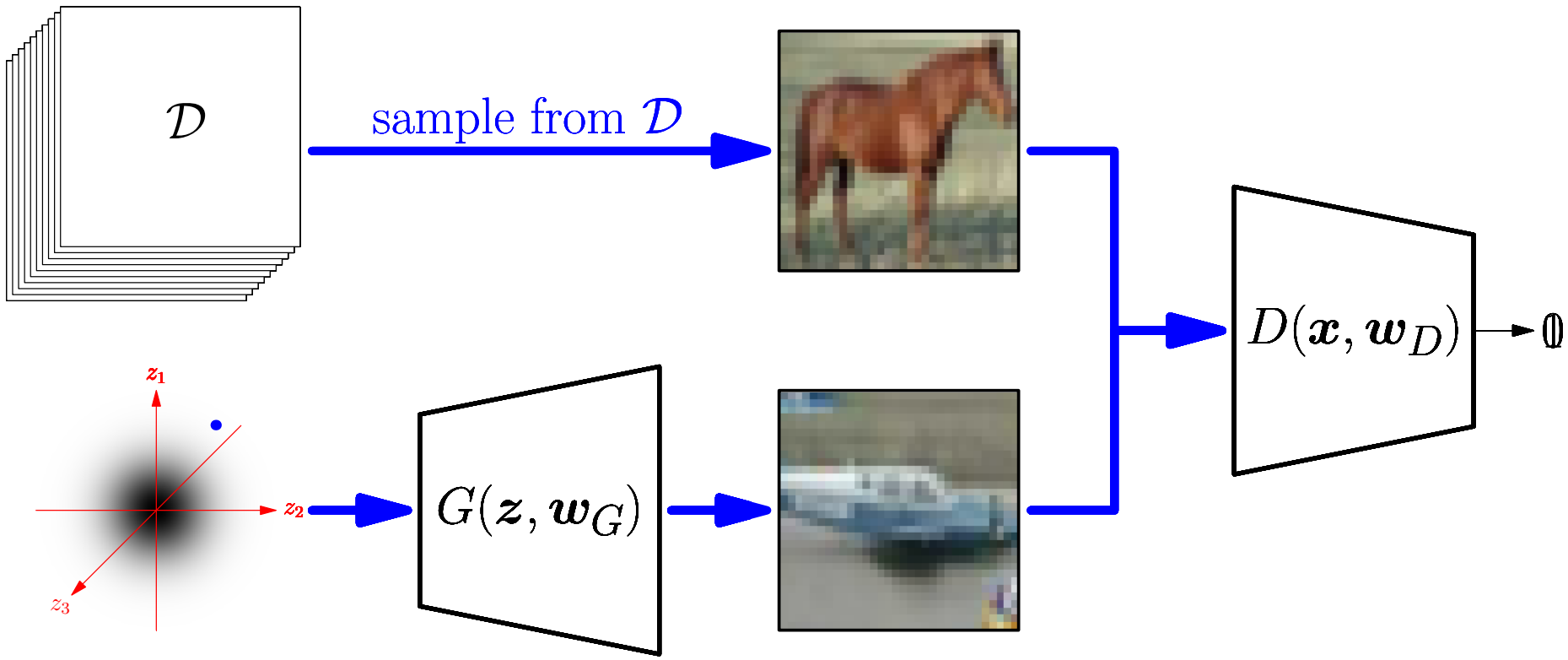


Outline

1. Comparing Distributions
2. Optimal Transport
3. Duality
4. **Applications**



How GANs Work



Wasserstein GANs

- A GAN has a **generator** $G(\boldsymbol{z}, \boldsymbol{w}_G)$ that turns noise $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ into images, and a **discriminator** trained to tell them from real images
- GANs are hard to train: a discriminator that is too good gives the generator no useful gradient
- A Wasserstein GAN instead trains the generator to minimise W between the data distribution p and the generator's distribution q

$$\min_{\boldsymbol{w}_G} \max_{\alpha} \Big(\mathbb{E}_p[\alpha(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{z}}[\alpha(G(\boldsymbol{z}, \boldsymbol{w}_G))] \Big)$$

- The **critic** α is a second network in place of the discriminator; it still gives a gradient when the distributions do not overlap

The Critic

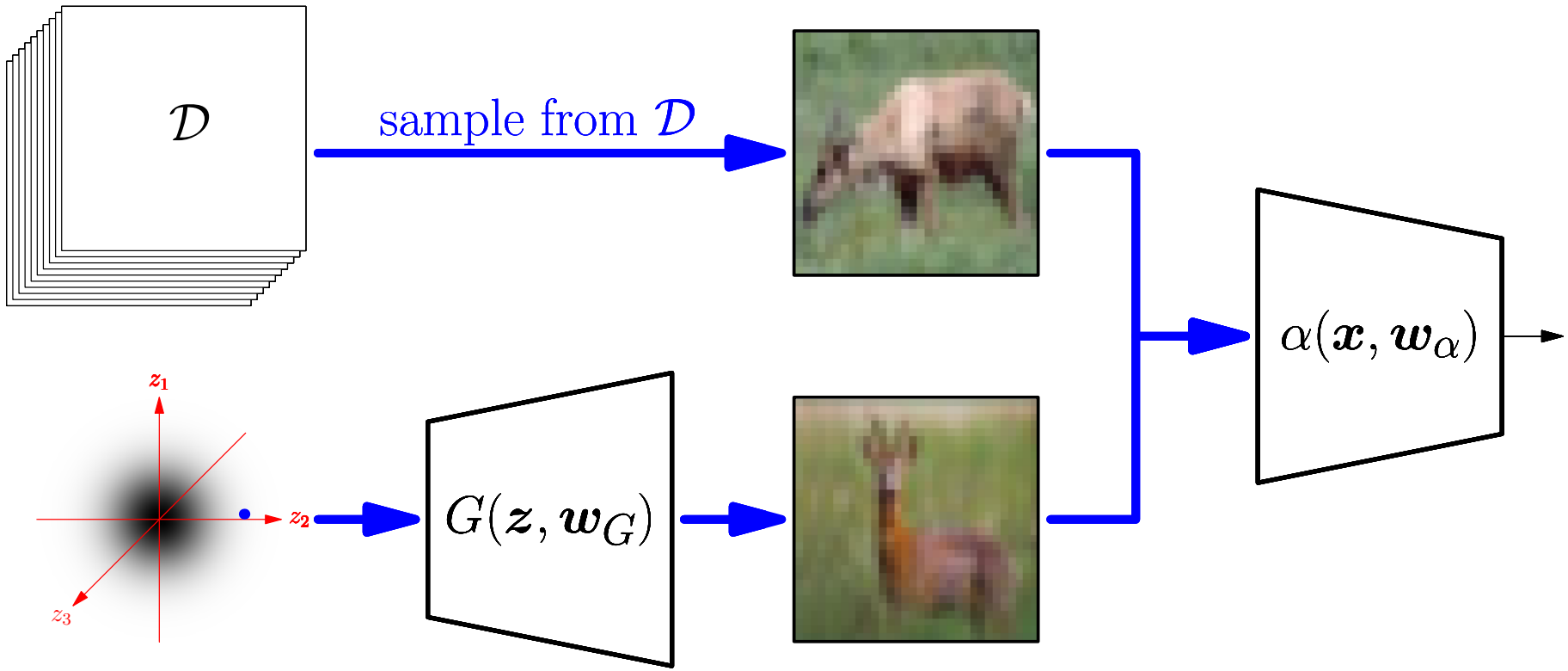
- Estimate the expectations from a minibatch $\mathcal{B}$ of real images and n generated ones

$$C = \frac{1}{|\mathcal{B}|}\sum_{\boldsymbol{x}\in\mathcal{B}} \alpha(\boldsymbol{x},\boldsymbol{w}_\alpha) - \frac{1}{n}\sum_{i=1}^{n} \alpha(G(\boldsymbol{z}_i,\boldsymbol{w}_G),\boldsymbol{w}_\alpha)$$

- The critic's weights $\boldsymbol{w}_\alpha$ are trained to maximise C; the generator's weights $\boldsymbol{w}_G$ to minimise it
- The critic should be 1-Lipschitz, which in practice is only roughly enforced: by clipping the weights (the original WGAN), penalising $\|\boldsymbol{\nabla}\alpha\|$ away from 1, or scaling each weight matrix to spectral norm 1

Wasserstein GANs

$$\max_{\boldsymbol{w}_\alpha} \min_{\boldsymbol{w}_G} \frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{x} \in \mathcal{B}} \alpha(\boldsymbol{x}, \boldsymbol{w}_\alpha) - \frac{1}{n} \sum_{i=1}^{n} \alpha(G(\boldsymbol{z}_i, \boldsymbol{w}_G), \boldsymbol{w}_\alpha)$$



Entropic Regularisation

- Solving the linear programme for n points costs roughly $O(n^3)$: too slow for large problems

- Cuturi (2013) added an entropy term, with $\epsilon > 0$

$$\min_{\gamma} \sum_{i,j} d_{ij}\gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij}$$

  with the same constraints

- The optimisation is given by: $\max_{\boldsymbol{\alpha},\boldsymbol{\beta}} \min_{\gamma} \mathcal{L}(\gamma, \boldsymbol{\alpha}, \boldsymbol{\beta})$

$$\mathcal{L} = \sum_{i,j} d_{ij}\gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij} - \sum_i \alpha_i \left(\sum_j \gamma_{ij} - p_i \right) - \sum_j \beta_j \left(\sum_i \gamma_{ij} - q_j \right)$$

Solving the Primal Problem

- Using

$$\mathcal{L} = \sum_{i,j} d_{ij}\gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij} - \sum_i \alpha_i \left(\sum_j \gamma_{ij} - p_i \right) - \sum_j \beta_j \left(\sum_i \gamma_{ij} - q_j \right)$$

- Setting the derivative of the Lagrangian to zero gives

$$\gamma_{ij} = \mathrm{e}^{(\alpha_i - d_{ij} + \beta_j)/\epsilon - 1} = u_i K_{ij} v_j$$

where $u_i = \mathrm{e}^{\alpha_i/\epsilon - 1}$, $v_j = \mathrm{e}^{\beta_j/\epsilon}$ and $K_{ij} = \mathrm{e}^{-d_{ij}/\epsilon}$

- We have to choose u_i and v_j (equivalently the Lagrange multipliers) so that

$$\sum_j u_i K_{ij} v_j = p_i \qquad \sum_i u_i K_{ij} v_j = q_j$$

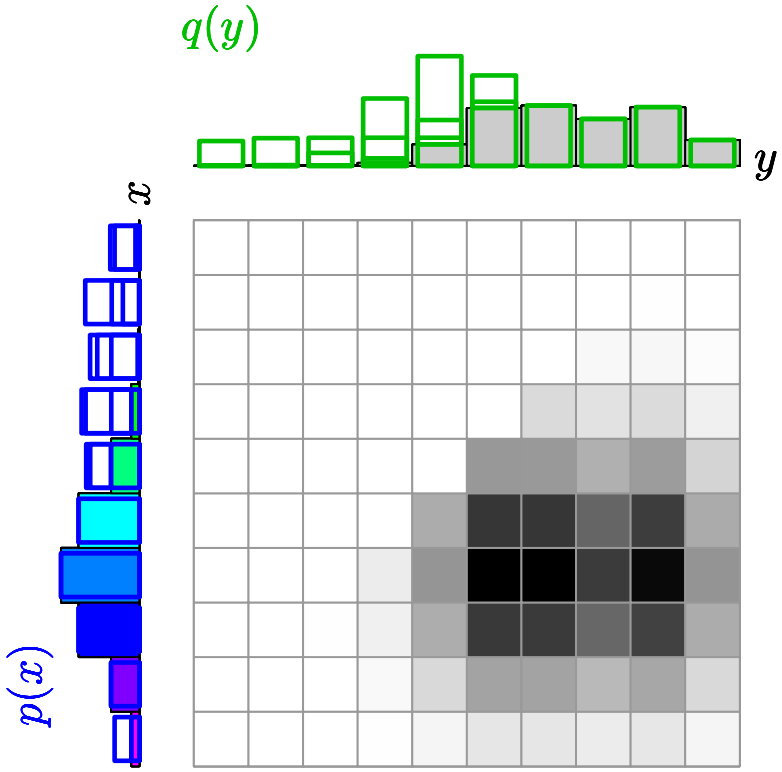
The Sinkhorn Algorithm

- Choose $\boldsymbol{u}$ and $\boldsymbol{v}$ so the row and column sums are $\boldsymbol{p}$ and $\boldsymbol{q}$: alternately rescale ($\oslash$ is element-by-element division)

$$\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (\mathbf{K}^{\mathsf{T}}\boldsymbol{u}), \qquad \boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (\mathbf{K}\boldsymbol{v})$$

- This is Sinkhorn's matrix-scaling algorithm: each step is two matrix–vector products
- Fast on GPUs and differentiable, so it can sit inside a loss
- As $\epsilon \to 0$ we recover the exact transport cost; larger ϵ is faster but blurs the plan
- This is how optimal transport is mostly computed in ML today

Sinkhorn in Action



iteration 1 $\mathbf{K}$ $\boldsymbol{q}$

exact Wasserstein distance $= 3.460$

$$\gamma = \mathrm{diag}(\boldsymbol{u})\,\mathbf{K}\,\mathrm{diag}(\boldsymbol{v})$$

Evaluating Generative Models: FID

- For Gaussians, W_2 has a closed form

$$W_2^2 = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 + \mathrm{Tr}\Big(\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2 - 2\big(\boldsymbol{\Sigma}_1\boldsymbol{\Sigma}_2\big)^{1/2}\Big)$$

  (in one dimension $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$)

- The **Fréchet Inception Distance** (Heusel et al., 2017) fits Gaussians to a pretrained network's features of real and of generated images, and reports this distance

- It is the standard score for image generators, GANs and diffusion models alike (lower is better)

- Caveat: it only compares means and covariances of the features

Lessons

- The Wasserstein distance compares distributions by how far probability mass must move, so it behaves well when they do not overlap
- Computing it is a linear programme, whose dual is a maximum over 1-Lipschitz functions
- That dual turned GANs into Wasserstein GANs: an elegant piece of theory
- In practice optimal transport is computed with Sinkhorn, and the W_2 distance between Gaussians is the FID used to evaluate generative models